

school_app

Missing Features Roadmap

15 high-leverage gaps to ship next

Generated 2026-05-24

Each feature includes a ready-to-paste prompt starting with `/graphify`

Generated: 2026-05-24 · Target audience: app owner planning the next 3–6 months of work

Each feature below has:

- **Why** — the user/business impact (focused on Indian K-12 reality)
 - **Effort** — rough engineering estimate
 - **Ready-to-paste prompt** — starts with `/graphify` so Claude Code first maps the relevant area before building. Copy the entire fenced block into Claude Code.
-

1. Real push notifications (FCM)

Why: Today `NotificationService` only writes documents to Firestore. Guardians and teachers must open the app to discover absences, leave approvals, or announcements. In an Indian school, parents check phones; they don't pull-to-refresh. Real push is the single highest-leverage missing feature.

Effort: 2–3 days. `firebase_messaging` plugin + Cloud Function trigger on `notifications/*` create + per-device FCM-token storage in `allowed_users`.

`/graphify lib/services/notification_service.dart` and every screen that calls it.

Then implement Firebase Cloud Messaging end-to-end:

1. Add `firebase_messaging` to `pubspec.yaml`.
2. Request notification permission on first run after role login.
3. Store the device FCM token under `allowed_users/{userId}.fcmTokens[]` (array, so a user can have multiple).
4. Add a Cloud Function (in `functions/`, init with ``firebase init functions`` if absent) that triggers on `notifications/*` create.
5. Handle foreground/background/terminated open routing so tapping a notification deep-links to the relevant screen.
6. Update the consent flow to mention push permission.

Show me the diff before applying.

2. Online fee payment (Razorpay / UPI)

Why: `FeeService` and `FeeCollectionScreen` look like manual cash-marking. In 2026 most Indian parents pay via UPI; collecting cash in the office is the actual operational bottleneck. Wire Razorpay so parents pay inside the Guardian dashboard and a receipt PDF auto-generates.

Effort: 4–5 days. Razorpay Flutter SDK + signed webhook on a Cloud Function to mark the fee as paid in Firestore (never trust the client).

`/graphify lib/services/fee_service.dart, lib/services/fee_reminder_service.dart, and lib/screens/fee_`
Then add Razorpay (`razorpay_flutter`) integration:

1. New "Pay Online" button in GuardianDashboard fee section that opens Razorpay checkout for outstanding
2. On payment success the client writes a pending receipt; a Cloud Function verifies the Razorpay signature
3. Generate a PDF receipt using the existing ``pdf`` package (model after `lib/utils/report_card_pdf_builder`)
4. Add UPI deep-link fallback (`intent://`) for users without Razorpay account.

Never trust the Flutter client to confirm payment – only the webhook.

Show me the migration path for in-flight unpaid records.

3. Bus / transport tracking

Why: Indian school buses are a recurring parent anxiety. Even a coarse "bus has crossed Gate 3" stage list beats nothing. Optional GPS later.

Effort: 3–4 days for stage-based; +1 week for live GPS.

/graphify the existing student model and guardian_dashboard.dart to understand where transport info w
Then design a Transport module:

1. New `routes` collection: {routeId, name, stops:[{name, sequence, expectedTime}]}
 2. Assign students to routes (Student model gains routeId).
 3. Driver/Conductor role app screen with a "Mark stop X reached" button that writes to routes/{routeI
 4. Guardian dashboard shows a live "Bus is at: \${currentStop}" card with an estimated-arrival countdo
 5. Stretch goal: optional live GPS via geolocator plugin for active routes – toggle in settings, not
Pick stage-based first; do not start with GPS.
-

4. Multi-language support (i18n)

Why: Only privacy notices are bilingual today. Many guardians in Tier-2/3 cities read only Hindi or a regional language. The teacher app could stay English; the **guardian app** must speak Hindi at minimum, ideally also Marathi/Tamil/Telugu/Bengali for the markets you care about.

Effort: 3 days for the framework + 2 days/language for translation.

/graphify lib/screens/guardian_*.dart and lib/utls/privacy_notice.dart (which already does bilingual)
Then convert the guardian-facing portion of the app to use flutter_localizations + intl ARB files:

1. Add flutter_localizations and intl to pubspec.yaml.
2. Configure MaterialApp.supportedLocales = [en, hi] in main.dart.
3. Extract every hardcoded user-facing string in guardian_dashboard.dart, attendance_views the guardian...
4. Generate the en.arb and hi.arb in lib/l10n/.
5. Add a language toggle in the guardian profile area, persisted in SharedPreferences.

Do NOT translate teacher/coordinator/principal screens in this pass – scope must stay tight.
List every untranslated string you find so I can review before translation.

5. WhatsApp Business API integration

Why: Today the app builds a `wa.me/...` URL and opens WhatsApp manually (see `_WhatsAppNotifySheet` in `attendance_screen.dart`). That requires the teacher to tap "Send" for every parent — fine for 5 absentees, awful for a class trip permission slip blast. WhatsApp Business Cloud API (\$0.005/msg for India) makes it server-side.

Effort: 3 days + Meta app verification (1–2 weeks externally, separate track).

/graphify the `_WhatsAppNotifySheet` flow and `lib/services/notification_service.dart`.

Then design a server-side WhatsApp Business API sender:

1. New Cloud Function `sendWhatsAppTemplate(audience, templateId, params)` calling Meta Graph API /me
 2. Store WhatsApp template definitions in Firestore (`wa_templates/{id}`) with English + Hindi variants
 3. Replace the manual `wa.me` loop in `_WhatsAppNotifySheet` with a single "Send via WhatsApp" button that
 4. Capture delivery + read receipts via the Meta webhook and write back to `notifications/{id}.delivery`
 5. Keep `wa.me` as the local fallback when the school hasn't onboarded WABA yet — feature-flag in settings
- Cost optimization: batch sends so multiple absentees in one class = one API call when possible.
-

6. Dark mode

Why: The deleted `ThemeProvider` indicates this was attempted and abandoned. Bring it back properly — `AppTheme` is already the single source of truth, so it's mostly plumbing.

Effort: 1.5 days.

/graphify lib/theme.dart and every screen that reads from AppTheme.

Then add a proper dark theme:

1. Add `AppTheme.darkPrimary`, `darkBackground`, `darkSurface`, `darkOnSurface` constants alongside the exist
 2. Build `ThemeData lightTheme()` and `ThemeData darkTheme()` factories.
 3. `MaterialApp` gets `themeMode` driven by a `ChangeNotifier` persisted to `SharedPreferences` ('theme_mode')
 4. Settings screen gets a three-way toggle (System / Light / Dark).
 5. Audit every raw `Color(0x...)` or `Colors.white`/`Colors.black` in `lib/screens/` — those break dark
- Project rule (see CLAUDE.md): never use raw color literals — surface every offender.
-

7. Biometric / PIN lock on app open

Why: The app holds attendance, fee, and medical data for hundreds of children. Anyone who picks up an unlocked teacher's phone can browse it. A 4-digit PIN or Touch/Face ID gate is table stakes for a DPDP-compliant school app.

Effort: 1 day.

/graphify lib/main.dart (`_SplashGate`) and lib/services/auth_service.dart.

Then add an app-lock layer:

1. Use `local_auth` plugin for biometric and a 4-digit PIN fallback (stored as bcrypt hash in `SharedPreference`).
2. New `LockGate` widget wraps `_SplashGate`'s output. On every cold start AND every resume-after-N-minutes.
3. Settings screen: "App lock" with timeout options (immediate, 1 min, 5 min, 30 min, off). Default 5 min.
4. Add a one-time PIN setup flow on first login after this ships.
5. Do NOT gate the role-selection screen itself – only post-login content.

Verify on iOS that `local_auth` Face ID prompt fires; macOS skips biometric and goes straight to PIN.

8. Bulk CSV import for students and teachers

Why: `pubspec.yaml` already has the `csv` dep but the import flow isn't surfaced. At admission season every school adds 100–500 students; manual entry is the #1 onboarding blocker.

Effort: 2 days.

/graphify lib/screens/add_student_screen.dart and lib/screens/teacher_management_screen.dart, plus li
Then build a Bulk Import flow:

1. New screen `BulkImportScreen` reachable from Admin and Coordinator dashboards.
 2. Download template button → produces a sample CSV with required columns (name, roll, className, se
 3. file_picker → parse CSV → preview table showing valid + invalid rows with inline error messages (
 4. Confirm → batch-write to Firestore in chunks of 500 (Firestore batched-write limit).
 5. Same flow for teachers.
 6. Write an entry to audit_log_service per import: who, when, file hash, row count.
- Critical: dry-run validation must complete before any write. Do not partially import.
-

9. Health records & medical alerts

Why: When a student faints on the playground, the staff member needs blood type, allergies, and an emergency contact in **two taps**, not buried under three screens. This is a real-world incident the app should prepare for.

Effort: 2 days.

/graphify lib/models/student.dart and lib/screens/student_details_screen.dart.

Then add a Health Profile sub-model:

1. New collection students/{id}/health/profile: bloodGroup, allergies[], chronicConditions[], medicat
2. Guardian app: edit-only access to their child's health profile (this is sensitive – write a tight
3. Teacher view: read-only with a prominent RED "MEDICAL ALERT" badge on the student row if allergies
4. Class teacher gets a one-tap "Emergency Card" PDF for any student (model after attendance certific
5. Update the consent screen – health data needs an explicit additional opt-in under DPDP.

Audit: every read of the health profile must write to audit_logs.

10. Transfer Certificate (TC) & Bonafide PDF generation

Why: When a student leaves, the school is legally required to issue a Transfer Certificate. Today this is typed in MS Word, signed, scanned. The data already lives in the app — generate the official PDF.

Effort: 2 days (template design + plumbing).

/graphify lib/utils/report_card_pdf_builder.dart and lib/services/report_card_template_service.dart.
Then add two new PDF generators in lib/utils/:

1. tc_pdf_builder.dart – Transfer Certificate matching the standard CBSE/state-board format: photo, a
 2. bonafide_pdf_builder.dart – short certificate confirming the student is enrolled (banks/visa offic
 3. Add an admin-only "Generate TC" action on the student detail screen → sequential certificate numb
 4. Audit log entry on issuance: cert number, issued to, issued by.
 5. Optional: digital signature placeholder image stored per principal account.
- Verify the layout prints correctly on A4 in both portrait and landscape.
-

11. Crash reporting + product analytics

Why: When a teacher hits a bug at 7:50am during the attendance rush, you find out from a WhatsApp message, not from data. Firebase Crashlytics + Analytics gives you the stack trace and the funnel.

Effort: 1 day.

/graphify lib/main.dart to find the right place to initialize Firebase plugins.

Then wire Firebase Crashlytics + Firebase Analytics:

1. Add `firebase_crashlytics` and `firebase_analytics` to `pubspec.yaml`.
 2. In `main.dart`, after `Firebase.initializeApp`, register `FlutterError.onError` → Crashlytics, plus `Platform`.
 3. Log custom events for the 5 funnels that matter: `role_login`, `attendance_save`, `leave_submit`, `fee_pay`, `school_settings`.
 4. Set Crashlytics user identifier from `AuthService` session (`role + userId`, NEVER `name/phone` – privacy).
 5. Add a kill switch in `school_settings` so a single school can opt out.
- Do NOT enable Crashlytics in debug builds – gate with `kReleaseMode`.
-

12. Audit log viewer UI

Why: `audit_log_service.dart` exists (recently added) but has no viewer. Logs you can't read aren't logs. Principal/owner needs a searchable timeline for DPDP and internal accountability.

Effort: 1.5 days.

/graphify lib/services/audit_log_service.dart and every call site that writes to it.

Then build AuditLogViewerScreen accessible from PrincipalDashboard and OwnerHome:

1. Paginated list (latest first) showing: timestamp, actor (role + name), action, target entity (stud
 2. Filter chips: date range, actor role, action type, entity type.
 3. Tap a row → detail sheet with the full before/after JSON diff (use `json_diff` package or hand-roll
 4. Export filtered range as CSV (use the `csv` package already in pubspec).
 5. Firestore index probably needed on (schoolId, timestamp) – flag any missing indexes you encounter.
- This view must be read-only – never expose delete or edit.
-

13. Internal staff chat / DM

Why: Announcements are broadcast-only. When the principal needs to ask the Math HOD a quick question, today they use a separate WhatsApp group — losing the audit trail. A simple thread-per-staff-pair chat keeps it inside the app.

Effort: 3–4 days for MVP (Firestore-backed, no presence).

/graphify lib/services/notification_service.dart, lib/screens/announcements_screen.dart, and the existing

Then design a Staff Chat module:

1. New collection `chats/{chatId}` where chatId is the sorted pair of userIds, e.g., u123_u456.
2. Sub-collection messages/{msgId}: senderId, text, attachments[], timestamp, readBy[].
3. List of conversations on the home screen for teacher/coordinator/principal – sorted by latest message.
4. Tap → ChatThreadScreen with text input + image attachment (gallery_service can be reused for uploads).
5. Push notification on incoming message (depends on Feature 1 shipping first).

Scope rules:

- No group chats in v1 – DM only.
 - No typing indicators or presence – too expensive on Firestore reads.
 - Guardian users CANNOT chat with staff (use leave/complaint flows instead).
-

14. Comprehensive test coverage

Why: `test/` has effectively a stub. The attendance and fees code paths are the highest-risk: corrupted attendance data is impossible to reconstruct, mis-applied fees create parent complaints. Unit + integration tests give you the confidence to refactor (Phase 2 reorg can't happen safely without them).

Effort: 1 week for the critical-path 60% coverage.

`/graphify lib/services/` to map every public method and its callers.

Then write a test suite that prioritises the high-risk services first:

1. `test/services/student_service_test.dart` – `getStudents`, `addStudent`, attendance roundtrip with Firestore
2. `test/services/timetable_service_test.dart` – clash detection, settings cache invalidation.
3. `test/services/fee_service_test.dart` – partial payment math, late-fee calc.
4. `test/services/offline_queue_service_test.dart` – queue ordering, dedup-by-key (`className+dateKey`).
5. `test/services/audit_log_service_test.dart` – append-only invariant (never mutates).
6. Integration: `test/widget/attendance_flow_test.dart` – full Take Attendance → Save → Sync → Notify

Use `firebase_authmocks` + `fake_cloud_firestore` packages, NOT real Firebase, in tests.

Target: 60% line coverage on `lib/services/`, 0% required on screens (UI changes too fast).

15. Backup + restore for school data

Why: A coordinator deleting a class by accident, or a Firestore migration going wrong, has no undo today. A nightly export to Cloud Storage per school, retained 30 days, is a 2-hour insurance policy.

Effort: 1 day (Cloud Function + tiny admin UI).

/graphify the firestore.rules and lib/services/school_service.dart to understand multitenant boundaries. Then add a backup pipeline:

1. Scheduled Cloud Function (cron: nightly 2:00 AM IST) per school: export schools/{schoolId}/{studentId} to bucket.
 2. Lifecycle rule on the bucket: delete after 30 days.
 3. Owner home: "Backups" tab listing recent backups with size + download link (signed URL, 1-hour expiry).
 4. Manual "Backup now" button on the same tab (rate-limit to 1/hour per school).
 5. Document the restore procedure in docs/ — do NOT build a UI-driven restore in v1; restores happen via CLI. This must be opt-in per school (settings/main.backupsEnabled, default true for paid tiers).
-

Appendix — features deliberately NOT in this list

- **Library, inventory, lesson plans, online quizzes, live-class integration** — all valuable but each is its own product. Don't bloat the core app before the 15 above are solid.
- **Multi-school owner UI polish** — the `owner/` screens exist; treat as separate hardening pass.
- **Guardian online homework submission** — wait until WhatsApp + push are done; otherwise guardians won't notice the homework was set.
- **Timetable PDF "class view"** — `timetable_screen.dart` (zero refs) was kept by the cleanup pass for exactly this reason. Wire it in or delete it.